NAME:RITHIKA P
REG NO:192565002
CODE:CSA0406
COURSE:OPERATING SYSTEMS

Q1) Deadlock bugging
Problem Identification
The deadlock occurs because:
P1 locks R1 and waits for R2.
P2 locks R2 and waits for R1.
Each process holds one resource while waiting for the other.
Neither process releases its resource, causing an infinite wait.
This is a circular wait, one of the four necessary conditions for deadlock.
EXECUTION SEQUENCE:
P1 locks R1 successfully.
P2 locks R2 successfully.
P1 requests R2 but waits because P2 holds it.
P2 requests R1 but waits because P1 holds it.
Both processes wait for each other indefinitely, resulting in a deadlock.
OPTIMIZATION:
Use resource ordering.
Both processes should acquire resources in the same order.
Process P1
lock(R1)
lock(R2)
Critical Section
unlock(R2)
unlock(R1)

Process P2
lock(R1)
lock(R2)
Critical Section
unlock(R2)
unlock(R1)
ANALYSIS:Since every process requests resources in the same order, circular waiting is removed and deadlock cannot occur.

Q2)CPU Utilization Problem
Problem Identification:
CPU utilization is only 30% because processes frequently move:
Ready → Waiting → Ready → Waiting
This indicates the processes are I/O-bound rather than CPU-bound.
The CPU remains idle while waiting for I/O operations.
DEBUGGING:
Frequent waiting states suggest:
- Continuous disk/network access
- Short CPU bursts
- High I/O waiting time

As a result:
- CPU executes briefly.
- Process waits for I/O.
- CPU becomes idle.

OPTIMIZATION:
Use asynchronous I/O
- CPU continues executing other tasks while I/O completes.

Increase multiprogramming
- Schedule another ready process whenever one enters waiting state.

ANALYSIS:Better scheduling ensures another process uses the CPU instead of leaving it idle, increasing CPU utilization.

Q3)Memory Bottleneck Debugging
Problem Identification:
Memory usage:
- Code = 5 MB
- Heap = 150 MB
- Stack = 1 MB × 25 = 25 MB

Total memory used:
5 + 150 + 25 = 180 MB

Although RAM is 2 GB, the large heap and many threads can cause poor memory locality.
Frequent page faults occur because memory pages are repeatedly swapped between RAM and storage.
DEBUGGING:
Main issue:
    Large heap accesses
    Multiple thread stacks
    Poor cache locality
    Increased page faults
    Optimization
Optimization 1
    Reduce heap usage by releasing unused memory.
Optimization 2
    Reduce the number of threads or use a thread pool.
ANALYSIS:Smaller memory usage reduces page faults and improves execution speed.

Q4) Starvation Debugging
Problem Identification
Queue structure:
Q1 → Interactive
Q2 → System
Q3 → Batch (P5)
Scheduler always serves higher-priority queues first.
If Q1 and Q2 continuously receive processes, P5 in Q3 never executes.
This is starvation.
DEBUGGING
Execution:
Q1 arrives
↓
CPU executes Q1

More Q1 arrives
↓

Q2 arrives
↓

P5 remains waiting
↓

Infinite postponement
OPTIMIZATION:
Use Aging and increase the priority of P5 gradually as its waiting time increases.
Example:
After every 20 ms waiting
Priority++
Eventually
Q3 → Q2 → Q1
ANALYSIS:Aging guarantees every process eventually receives CPU time and prevents starvation.

Q5)Context-Switch Overhead Debugging
Problem Identification:
Given:
Context switches = 20,000/sec
Burst time = 2 ms
Time quantum = 1 ms
Very frequent I/O
Since quantum (1 ms) is smaller than burst time (2 ms), each thread is preempted before finishing.
Frequent I/O also causes additional context switches.
DEBUGGING:
Flow:

Run 1 ms
↓
Quantum expires
↓
Context Switch
↓
Another thread
↓

I/O wait
↓
Context Switch
Thousands of switches occur every second.
OPTIMIZATION:
Optimization 1:
Increase time quantum.
Example:
1 ms → 4 ms
Threads complete before preemption.
Optimization 2
Reduce unnecessary I/O waits by using:
I/O buffering
Asynchronous I/O
ANALYSIS:A larger time quantum reduces scheduler interruptions, while optimized I/O lowers waiting-related context switches, significantly improving system performance.